Practical: Problem Solving Through Search Algorithms

Name: Vedangi Eknath Patil

PRN: 202401040058

DIV: A

Roll No.: 19

Hospital Medical Sample Transport Route Planner

This practical implements a real-life **Hospital Medical Sample Transport Route Planner** in which a medical sample needs to be transported from a **Collection Room** to a **Pathology Lab** using different Artificial Intelligence search techniques.

The hospital is represented as a weighted graph containing different hospital locations such as the Collection Room, Reception, Emergency Desk, Storage, Waiting Area, Nurse Station, ICU Corridor, Pharmacy, Lab Corridor, Operating Wing, Radiology, and Pathology Lab.

Each connection between two hospital locations has an associated **transport time/cost**.

The program implements **uninformed search, informed search, local search, and constraint satisfaction techniques** to find or optimize a feasible route for transporting a medical sample.

The search algorithms considered in this practical are:

- Breadth First Search (BFS)
- Depth First Search (DFS)
- Depth Limited Search (DLS)
- Iterative Deepening Search (IDS)
- Bidirectional Search
- Greedy Best-First Search
- A* Search
- Hill Climbing
- Genetic Algorithm
- Backtracking Constraint Satisfaction

The algorithms are compared based on **transport cost, nodes/steps explored, search behavior, and ability to find a feasible route**.

---

1. Aim

To design and implement a **Hospital Medical Sample Transport Route Planner** using various Artificial Intelligence search algorithms such as BFS, DFS, DLS, IDS, Bidirectional Search, Greedy Best-First Search, A* Search, Hill Climbing, Genetic Algorithm, and Backtracking CSP, and compare their performance in finding an efficient and feasible route from the **Collection Room** to the **Pathology Lab**.

---

2. Objectives

The objectives of this practical are:

- To understand the concept of state-space search in Artificial Intelligence.
- To model a hospital environment as a graph-based search space.
- To represent hospital locations and transport connections using a graph.
- To assign transport time/cost to connections between hospital locations.
- To implement different uninformed search algorithms.
- To implement informed search algorithms using heuristic information.
- To implement local search techniques for route optimization.
- To formulate hospital navigation restrictions as a Constraint Satisfaction Problem.
- To find a feasible route between the Collection Room and Pathology Lab.
- To calculate the transport cost of the discovered route.
- To calculate and compare the number of nodes/steps explored by different algorithms.
- To understand the difference between uninformed and informed search.
- To understand the advantages and limitations of local search.
- To demonstrate the role of constraints in hospital sample transportation.
- To compare the performance of different search strategies.

---

3. Application Details

Application Name

Hospital Medical Sample Transport Route Planner

Real-Life Problem

Consider a hospital where medical samples such as **blood, urine, tissue, or other laboratory specimens** need to be transported from a **Collection Room** to the **Pathology Lab**.

The hospital contains different departments and corridors. The sample transport system needs to select a route through these locations.

The hospital environment contains:

- Collection Room
- Reception
- Emergency Desk
- Waiting Area
- Nurse Station
- Storage
- Pharmacy
- ICU Corridor
- Operating Wing
- Lab Corridor
- Radiology
- Pathology Lab

The robot or transport system starts from the **Collection Room** and has to reach the **Pathology Lab**.

The objective is to find a suitable route while considering the transport cost or time between locations.

The hospital is represented using a **weighted graph**.

For example:

```
Collection Room → Storage = 4 units

Storage → Pharmacy = 3 units

Pharmacy → Lab Corridor = 4 units

Lab Corridor → Pathology Lab = 4 units
```

Therefore:

```
Collection Room
       ↓
    Storage
       ↓
    Pharmacy
       ↓
  Lab Corridor
       ↓
 Pathology Lab
```

has a total transport cost of:

```
4 + 3 + 4 + 4 = 15 units
```

The program stores hospital locations and transport costs in a graph representation.

---

4. Hospital Map

The simplified hospital environment used in this practical is:

```
                HOSPITAL SAMPLE TRANSPORT MAP


 Collection Room
    |      |       |
 Reception Emergency Storage
    |       |       |
 Waiting  Nurse    Pharmacy
    |       |        |
    +-------+---- ICU Corridor
                  |       |
              Operating  Lab Corridor
                 |       |      |
              Radiology--+------+
                    |       |
                    +-- Pathology Lab
```

```
START : Collection Room
GOAL  : Pathology Lab
```

The important locations are represented as nodes and the connections between them represent possible movement paths.

The edge value represents the **transport time/cost** between two locations.

For example:

```
Collection Room → Reception = 3
Collection Room → Emergency Desk = 5
Collection Room → Storage = 4
Storage → Pharmacy = 3
Pharmacy → Lab Corridor = 4
Lab Corridor → Pathology Lab = 4
Radiology → Pathology Lab = 5
```

The starting location is:

```
START = Collection Room
```

The target location is:

```
GOAL = Pathology Lab
```

---

5. Graph Representation

The hospital is represented using a Python dictionary.

Each hospital location is a node and the connected locations are represented as neighboring nodes.

For example:

```
"Collection Room": {
    "Reception": 3,
    "Emergency Desk": 5,
    "Storage": 4
}
```

This means that:

```
Collection Room → Reception = 3 units

Collection Room → Emergency Desk = 5 units

Collection Room → Storage = 4 units
```

The graph is weighted because every connection has a transport cost.

The complete graph contains locations such as:

- Collection Room
- Reception
- Emergency Desk
- Storage
- Waiting Area
- Nurse Station
- ICU Corridor
- Pharmacy
- Lab Corridor
- Operating Wing
- Radiology
- Pathology Lab

The program uses this graph for all search algorithms.

---

6. Algorithms Used

The practical contains **four major categories of search strategies**.

A. Uninformed Search Algorithms

Uninformed search algorithms do not use additional heuristic information about how close a location is to the target.

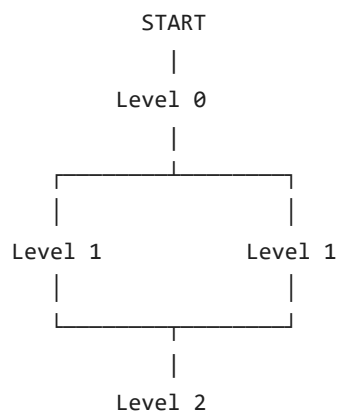
The algorithms used are:

1. Breadth First Search
2. Depth First Search
3. Depth Limited Search
4. Iterative Deepening Search
5. Bidirectional Search

---

1. Breadth First Search — BFS

BFS explores hospital locations level by level.



```
                    |
                  GOAL
```

Principle

- Uses a **Queue**.
- Follows **FIFO**.
- Explores nodes closest to the starting location first.

Data Structure

Queue using Python's:

```
deque
```

Basic Algorithm

```
BFS(Start, Goal)

1. Insert Start into Queue.

2. Mark Start as visited.

3. While Queue is not empty:

      a. Remove a node from the front.

      b. If node is Goal:
             return path.

      c. Add unvisited neighboring locations to Queue.

4. If Queue becomes empty:
      return failure.
```

The implementation uses Python's `deque` to implement the queue.

Hospital Application

BFS can be used when the hospital is represented as an unweighted graph and each movement is considered to have approximately equal cost.

BFS can find a route with the minimum number of edges under these conditions.

However, because the given hospital graph contains different transport costs, BFS does **not necessarily find the minimum-cost route**.

---

2. Depth First Search — DFS

DFS explores one branch as deeply as possible before backtracking.

Principle

- Uses a **Stack**.
- Follows **LIFO**.
- Goes deep before exploring other branches.

Data Structure

Stack

The implementation uses a Python list as a stack.

Basic Algorithm

```
DFS(Start, Goal)

1. Insert Start into Stack.

2. While Stack is not empty:

      a. Remove the top node.

      b. Mark it as visited.

      c. If node is Goal:
             return path.

      d. Push unvisited neighboring locations.

3. If Stack becomes empty:
      return failure.
```

Hospital Application

DFS can be used to explore connected areas of a hospital and determine whether the Pathology Lab is reachable from the Collection Room.

However, DFS does not guarantee the shortest or lowest-cost route.

The route found can depend on the order in which neighboring locations are explored.

---

3. Depth Limited Search — DLS

DLS is a modified version of DFS in which a maximum search depth is specified.

In the given program, the depth limit is:

```
Depth Limit = 8
```

The algorithm cannot search beyond the specified depth.

Principle

```
DLS = DFS + Depth Limit
```

Basic Algorithm

```
DLS(Start, Goal, Limit)

1. Start at the initial location.

2. Search using DFS.

3. Keep track of the current depth.

4. If Goal is found:
      return path.

5. If depth reaches Limit:
      stop searching that branch.

6. Continue with other branches.
```

The program uses recursive search and keeps track of the current depth.

Hospital Application

DLS can be useful when the medical sample transport system has a maximum allowed number of movements or when searching only within a limited portion of the hospital.

An insufficient depth limit may cause the Pathology Lab to remain undiscovered.

---

4. Iterative Deepening Search — IDS

IDS repeatedly performs DLS with increasing depth limits.

```
DLS depth 0

      ↓

DLS depth 1

      ↓

DLS depth 2

      ↓

DLS depth 3

      ↓
```

```
DLS depth 4

      ↓

     ...

      ↓

    GOAL
```

Basic Algorithm

```
IDS(Start, Goal)

1. Set depth = 0.

2. Perform DLS(Start, Goal, depth).

3. If Goal is found:
      return path.

4. Increase depth.

5. Repeat until Goal is found.
```

The program repeatedly calls DLS with increasing depth limits.

Hospital Application

IDS can be useful when the depth of the required route is unknown but memory usage needs to remain relatively controlled.

It gradually increases the search depth until the Pathology Lab is reached.

---

5. Bidirectional Search

Bidirectional Search searches simultaneously from:

```
Collection Room → → → → → Pathology Lab

Pathology Lab → → → → → Collection Room
```

The two searches continue until they meet.

Principle

```
Forward Search  : Collection Room → ...

Backward Search : Pathology Lab → ...
```

```
                    ↓

                Meeting
```

Basic Algorithm

```
1. Start one BFS from Collection Room.

2. Start another BFS from Pathology Lab.

3. Expand both searches.

4. Check whether their visited nodes intersect.

5. When they meet:
      construct the complete route.
```

The program maintains separate forward and backward queues and parent relationships.

Hospital Application

Bidirectional Search can reduce the amount of search required when both the Collection Room and Pathology Lab are known.

It is particularly useful when the search depth is large and the graph can be searched effectively from both directions.

B. Informed Search Algorithms

Informed search algorithms use heuristic information to guide the search toward the target.

The heuristic represents an estimated remaining cost from the current hospital location to the Pathology Lab.

The algorithms used are:

1. Greedy Best-First Search
2. A* Search

---

6. Greedy Best-First Search

Greedy Best-First Search chooses the hospital location that appears closest to the Pathology Lab.

Formula

$$f(n) = h(n)$$

Where:

$$h(n) = \text{estimated cost from current location to Pathology Lab}$$

The program defines heuristic values such as:

```
Collection Room = 18
Reception = 16
Emergency Desk = 15
Storage = 14
Waiting Area = 13
Nurse Station = 12
ICU Corridor = 9
Pharmacy = 10
Lab Corridor = 5
Operating Wing = 7
Radiology = 4
Pathology Lab = 0
```

The goal has heuristic value:

```
h(Pathology Lab) = 0
```

Example

Suppose two locations have:

```
Lab Corridor      h(n) = 5

Radiology         h(n) = 4
```

Greedy Best-First Search prefers Radiology because its heuristic value is smaller.

Basic Algorithm

```
Greedy Best First Search

1. Put Start in Priority Queue.

2. Calculate h(n).

3. Select the node with smallest h(n).

4. Expand the node.

5. Add suitable neighboring locations.

6. Repeat until Goal is reached.
```

The implementation uses Python's `heapq` as a priority queue.

Hospital Application

Greedy Best-First Search can guide the medical sample transport system toward the Pathology Lab using estimated distance information.

However, it does not necessarily produce the lowest-cost route because it considers the estimated remaining cost but ignores the transport cost already travelled.

---

7. A-Star Search

A* combines:

- Actual transport cost already travelled.
- Estimated transport cost remaining.

Formula

$$f(n) = g(n) + h(n)$$

Where:

- **g(n)** = actual transport cost from Start to current node.
- **h(n)** = estimated cost from current node to Goal.
- **f(n)** = estimated total route cost.

For example:

```
Collection Room → Storage

g(Storage) = 4

h(Storage) = 14

f(Storage) = 4 + 14

         = 18
```

The node with the smallest `f(n)` is selected.

Basic Algorithm

```
A*(Start, Goal)

1. Put Start in Priority Queue.

2. Set g(Start) = 0.

3. Calculate f(n) = g(n) + h(n).

4. Select node with smallest f(n).

5. Expand that node.

6. Update costs of neighboring locations.
```

```
7. Repeat until Goal is reached.
```

The program maintains the best known transport cost using `best_g`.

It calculates:

```
f = g + h
```

Hospital Application

A* can be used for efficient medical sample transportation between hospital departments.

It considers both:

```
Transport cost already travelled
+
Estimated cost remaining
```

Therefore, it provides a more cost-aware search than Greedy Best-First Search.

When the heuristic is appropriate and admissible for the given cost model, A* can provide an optimal minimum-cost route.

C. Local Search

Local search algorithms work with candidate solutions and attempt to improve them rather than systematically exploring the complete search tree.

The algorithms used are:

1. Hill Climbing
2. Genetic Algorithm

---

8. Hill Climbing Algorithm

Hill Climbing is a local search algorithm used for solving optimization problems.

It starts with an initial route and repeatedly moves toward a better neighboring route.

```
            Initial Route

                  ↓

         Generate Neighbor

                  ↓

         Evaluate Neighbor

                  ↓
```

```
            Is it better?

            /          \

          Yes           No

           ↓             ↓

    Move to it          Stop

           ↓

         Repeat
```

In Our Application

Initial state:

A valid route from the Collection Room to the Pathology Lab.

Neighbor:

Another valid hospital route that differs from the current route.

Evaluation:

Total transport cost.

Objective:

Minimize medical sample transport cost.

Example

Suppose the initial route has:

```
Route 1:

Collection Room → Emergency Desk → Nurse Station
→ ICU Corridor → Lab Corridor → Pathology Lab

Cost = Higher Cost
```

The algorithm searches for another route with a lower transport cost.

For example:

```
Route 2:

Collection Room → Storage → Pharmacy
→ Lab Corridor → Pathology Lab

Cost = 15 units
```

If Route 2 has a lower cost than the current route, the algorithm moves to Route 2.

Basic Components

1. Initial State

The algorithm starts with a candidate route.

2. Neighbor Generation

Alternative valid routes are considered as neighboring solutions.

The program generates possible routes using `ALL_ROUTES`.

3. Evaluation Function

The quality of each route is measured using:

```
Cost = Total Transport Cost
```

4. Selection

A better route with lower cost is selected.

5. Termination

The algorithm stops when no better neighboring route is available or the iteration limit is reached.

The program allows a maximum of:

```
20 iterations
```

Advantages

- Simple to understand and implement.
- Uses relatively little memory.
- Can improve a solution quickly.
- Useful for route optimization.
- Can be adapted to hospital transportation planning.

Limitations

- Can get stuck at a local optimum.
- May encounter a plateau.
- Does not guarantee the global optimum.
- Result can depend on the initial route.
- It may not explore all possible routes systematically.

A possible improvement is **Random-Restart Hill Climbing**, where the algorithm begins from different initial routes.

---

9. Genetic Algorithm

A Genetic Algorithm (GA) is an optimization algorithm inspired by natural evolution.

Instead of maintaining only one route, GA maintains a population of candidate routes and improves them over multiple generations.

Basic Process

```
Population

     ↓

Fitness Evaluation

     ↓

Selection

     ↓

Crossover

     ↓

Mutation

     ↓

New Population

     ↓

Repeat
```

Genetic Algorithm for Hospital Medical Sample Transport

Each possible route from:

```
Collection Room → Pathology Lab
```

is treated as a candidate solution.

Important Terms in Genetic Algorithm

1. Population

A collection of candidate hospital routes.

```
Population:

Route 1
```

```
Route 2

Route 3

Route 4

Route 5

...
```

The program uses a population size of:

```
20 routes
```

2. Chromosome

One candidate route is called a chromosome.

For example:

```
Collection Room → Storage → Pharmacy
→ Lab Corridor → Pathology Lab
```

represents one possible chromosome.

3. Gene

A gene represents one hospital location within the chromosome.

```
Collection Room → Storage → Pharmacy → Lab Corridor → Pathology Lab

                  ↑

                Gene
```

4. Fitness Function

The fitness function measures how good a route is.

Since the medical sample should preferably travel using a shorter route:

$$Fitness = \frac{1}{Distance + 1}$$

In this practical, `Distance` represents the total transport cost.

For example:

```
Transport Cost = 15 units

Fitness = 1 / (15 + 1)

        = 1 / 16
```

Another route:

```
Transport Cost = 30 units

Fitness = 1 / (30 + 1)

        = 1 / 31
```

Therefore, the route with lower transport cost has higher fitness.

5. Selection

Better routes are selected as parents for creating the next generation.

For example:

```
Route A → 30 units

Route B → 25 units

Route C → 15 units

Route D → 35 units
```

Routes with better fitness have a greater chance of being selected.

The program ranks the population according to fitness and selects better routes as parents.

6. Crossover

Crossover combines portions of two parent routes to create a new candidate route.

The program identifies common locations between parent routes and attempts to combine the two routes.

The resulting route is checked using the `valid_path()` function.

If the generated child is invalid, one of the parent routes is used instead.

7. Mutation

Mutation introduces a small random change to a candidate route.

In the program, mutation can replace a child with another randomly selected valid route.

The mutation rate used is:

```
0.20
```

This means mutation is applied with a probability of 20%.

8. Generation

One complete cycle of:

```
Selection

    ↓

Crossover

    ↓

Mutation
```

produces a new generation.

The program runs the Genetic Algorithm for:

```
30 generations
```

The best route found during the generations is recorded.

Genetic Algorithm Steps

```
1. Generate an initial population.

2. Calculate the fitness of every route.

3. Select better routes.

4. Select parents.

5. Perform crossover.

6. Perform mutation.

7. Create a new population.

8. Evaluate the new population.

9. Repeat for a specified number of generations.

10. Return the best route found.
```

Genetic Algorithm Flow

```
                    START

                      ↓

             Generate Population

                      ↓
```



Advantages

- Works with multiple candidate solutions.
- Can explore a large search space.
- Useful for complex route optimization.
- Mutation provides additional exploration.
- Suitable for hospital transportation planning.
- Can improve routes over multiple generations.

Limitations

- May require many generations.
- Results depend on parameters.
- Uses randomness, so results may vary.
- Does not necessarily guarantee the global optimum.
- Route crossover must be carefully handled to preserve valid hospital paths.

D. Constraint Satisfaction Search

Hospital medical sample transportation may involve constraints in addition to route distance.

For example:

- The route should use valid hospital connections.
- The same hospital location should not be unnecessarily repeated.
- The route must remain within the defined hospital graph.
- The sample should eventually reach the Pathology Lab.
- Invalid or disconnected routes should not be selected.

These requirements can be represented using a **Constraint Satisfaction Problem (CSP)**.

---

10. Backtracking Constraint Satisfaction

Backtracking systematically assigns possible hospital locations and checks constraints.

```
Start

  ↓

Select next location

  ↓

Check constraints

  ↓

 ┌──────────────┐
 │              │
Valid          Invalid
 │              │
 ↓              ↓
Continue       Backtrack
 │
 ↓
Goal reached
```

Variables

The positions of the medical sample transport system can be represented as:

$$X_1, X_2, X_3, \ldots, X_n$$

Each variable represents a position of the sample during transportation.

Domains

Each variable can contain possible hospital locations.

```
Domain = {
Collection Room,
```

```
Reception,
Emergency Desk,
Storage,
Waiting Area,
Nurse Station,
ICU Corridor,
Pharmacy,
Lab Corridor,
Operating Wing,
Radiology,
Pathology Lab
}
```

Constraints

The implementation applies constraints such as:

$$X_i \neq X_j$$

for already visited locations, so that the route does not unnecessarily repeat a location.

The next location must also be a valid connection in the hospital graph.

Therefore:

$$X_{i+1} \in Neighbors(X_i)$$

The final location must satisfy:

$$X_n = Pathology\ Lab$$

The route must also start from:

$$X_1 = Collection\ Room$$

Basic Algorithm

```
Backtracking(Start, Goal)

1. Select the next possible hospital location.

2. Check whether the location satisfies all constraints.

3. If valid:
      continue with the next location.

4. If invalid:
      backtrack.

5. Continue until Goal is reached.

6. If no valid assignment exists:
      return failure.
```

The program uses recursion to perform backtracking.

It maintains:

```
path
visited
order
```

to keep track of the current route and visited locations.

Hospital Application

Backtracking can ensure that the selected route follows valid hospital connections and does not repeatedly visit already occupied locations.

It is useful when medical sample transportation has strict route constraints.

---

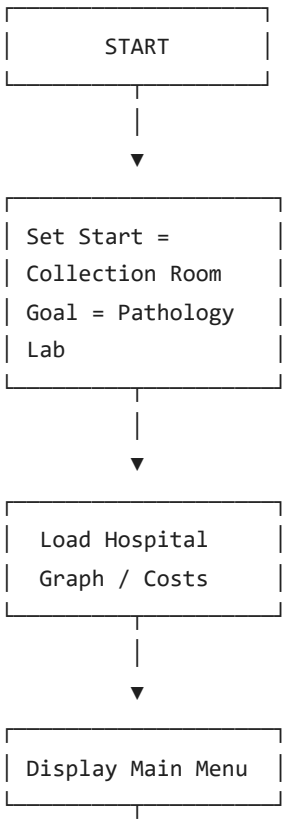
11. Difference Between Search Strategies

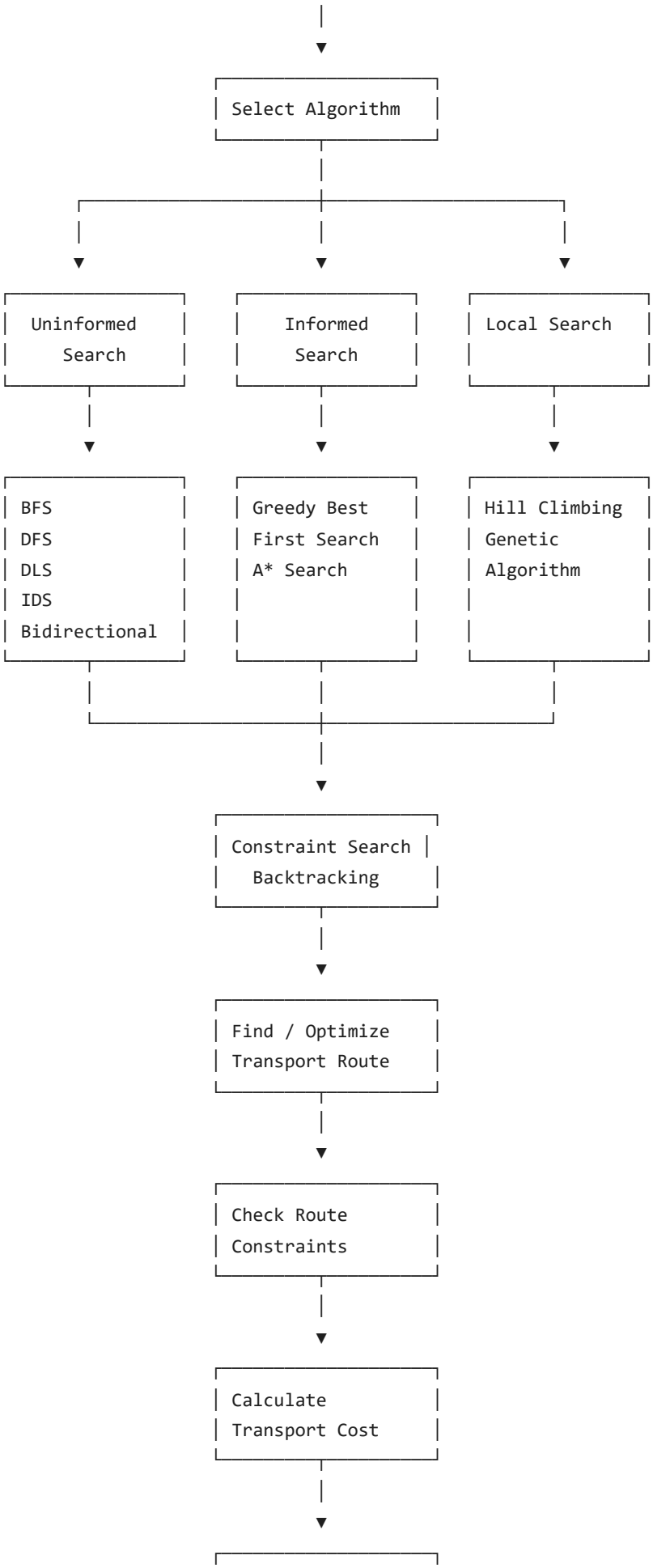
Uninformed Search	Informed Search	Local Search	
Does not use heuristic information	Uses heuristic information	Improves candidate solutions	Use
Explores based on graph structure	Uses graph + heuristic	Focuses on candidate routes	Che
Examples: BFS, DFS, DLS, IDS	Examples: Greedy, A*	Examples: Hill Climbing, GA	Exa
May explore many unnecessary states	Can guide search toward the goal	Does not systematically explore the complete search tree	Rej
Suitable for systematic route discovery	Suitable for goal-directed navigation	Suitable for optimization	Suit

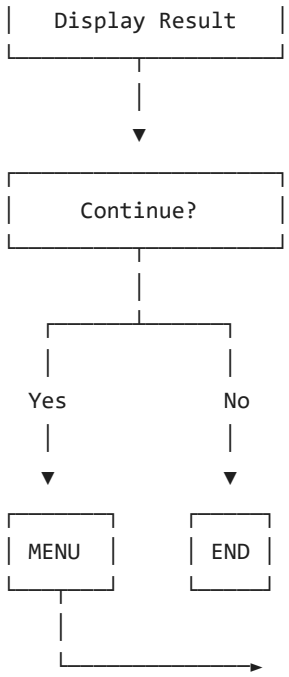
---

12. Overall Flowchart

The following flowchart represents the complete **Hospital Medical Sample Transport Route Planner**:




Select Algorithm
Uninformed Search
Informed Search
Local Search
BFS
DFS
DLS
IDS
Bidirectional
Greedy Best First Search
A* Search
Hill Climbing
Genetic Algorithm
Constraint Search Backtracking
Find / Optimize Transport Route
Check Route Constraints
Calculate Transport Cost



---

13. Detailed Program

The main program uses the hospital graph, transport costs, heuristic values, search functions, route evaluation functions, and an interactive interface.

The program creates an interactive **Hospital Medical Sample Transport Route Planner** using Python and `ipywidgets`.

The user can select a search algorithm and find or optimize a route between:

```
Collection Room → Pathology Lab
```

The program can display:

- Selected algorithm
- Generated route
- Transport cost
- Number of nodes/steps
- Search order
- Improvement history for Hill Climbing
- Number of generations for Genetic Algorithm
- Best route found by Genetic Algorithm
- Route feasibility through CSP
- Comparison of all algorithms

The implementation keeps each algorithm as a separate function so that the algorithms can be tested and compared independently.

The main interface provides:

```
RUN SEARCH
Compare Results
View Hospital Map
Clear
```

The user can select algorithms from the dropdown menu.

The available algorithms are:

```
BFS
DFS
DLS
IDS
Bidirectional
Greedy Best First
A*
Hill Climbing
Genetic Algorithm
Backtracking CSP
```

---

14. Observation

After executing the program, the following observations can be made.

BFS

BFS searches hospital locations level by level.

It can find a minimum-edge route when all movements have equal cost.

Since the hospital graph contains different transport costs, the BFS route is not necessarily the minimum-cost route.

DFS

DFS explores one branch deeply before backtracking.

The route found depends on the order in which neighboring hospital locations are explored.

DFS does not guarantee the lowest transport cost.

DLS

DLS searches only up to the specified depth limit.

In the program:

```
Depth Limit = 8
```

If the target is deeper than the allowed limit, the algorithm may fail to find it.

IDS

IDS gradually increases the depth limit:

```
1 → 2 → 3 → 4 → ...
```

until the Pathology Lab is found.

It combines repeated depth-limited searches with increasing depth.

Bidirectional Search

Bidirectional Search progresses from both:

```
Collection Room
```

and:

```
Pathology Lab
```

The two searches attempt to meet in the middle.

This can reduce the effective search depth for suitable graph problems.

Greedy Best-First Search

Greedy Best-First Search uses:

$$h(n)$$

to select the hospital location that appears closest to the Pathology Lab.

It can reach the goal efficiently in some cases, but the route is not necessarily minimum-cost.

A*

A* uses:

$$f(n) = g(n) + h(n)$$

Therefore, it considers both the transport cost already incurred and the estimated remaining cost.

With an appropriate admissible heuristic and cost model, A* can find a minimum-cost route.

Hill Climbing

Hill Climbing starts with a candidate route and attempts to improve it by selecting a lower-cost neighboring route.

It stops when no better neighboring route is available or the iteration limit is reached.

The program allows up to:

```
20 iterations
```

Genetic Algorithm

The Genetic Algorithm maintains a population of candidate routes.

It uses:

```
Selection
Crossover
Mutation
```

to improve the population over generations.

The program uses:

```
Population Size = 20

Generations = 30

Mutation Rate = 0.20
```

The best route found is displayed at the end.

Backtracking CSP

Backtracking checks whether each selected hospital location satisfies the route constraints.

If a location violates the constraints or creates an invalid route, the algorithm backtracks and tries another location.

---

15. Result Analysis

For the hospital graph used in this practical, one feasible route is:

```
Collection Room → Storage → Pharmacy → Lab Corridor → Pathology Lab
```

The transport costs are:

```
Collection Room → Storage = 4 units

Storage → Pharmacy = 3 units

Pharmacy → Lab Corridor = 4 units

Lab Corridor → Pathology Lab = 4 units
```

Therefore:

$$Total\ Cost = 4 + 3 + 4 + 4$$

$$Total\ Cost = 15\ units$$

Thus, the above route has a transport cost of:

```
15 units
```

Another feasible route is:

```
Collection Room → Reception → Waiting Area
→ Lab Corridor → Pathology Lab
```

Its cost is:

```
3 + 2 + 6 + 4 = 15 units
```

Therefore, multiple feasible routes may have the same transport cost.

Expected Comparison

Algorithm	Category	Route	Cost
BFS	Uninformed	Depends on traversal	Depends on result
DFS	Uninformed	Depends on traversal order	Depends on result
DLS	Uninformed	Depends on depth limit	Depends on result
IDS	Uninformed	Depends on search depth	Depends on result
Bidirectional	Uninformed	Depends on meeting point	Depends on result
Greedy Best-First	Informed	Depends on heuristic	Depends on result
A*	Informed	Cost-aware route	Depends on result
Hill Climbing	Local	Improved candidate route	Depends on result
Genetic Algorithm	Local/Optimization	Best route found	Depends on run
Backtracking	CSP	Feasible route	Depends on traversal

Note: The exact nodes/steps and route produced by each algorithm must be taken from the actual program output. They should not be manually assumed.

---

16. Performance Metrics

The algorithms can be compared using the following metrics.

1. Transport Cost

$$Cost = \sum Movement\ Costs$$

This represents the total transport cost/time required to move the medical sample from the Collection Room to the Pathology Lab.

2. Nodes/Steps Visited

The number of hospital states or locations examined by the algorithm.

The program records the search order and calculates:

```
Nodes/Steps = len(order)
```

For Hill Climbing, the number of improvement states is recorded.

For Genetic Algorithm, the number of generations is recorded.

3. Path Length

The number of movements from the Collection Room to the Pathology Lab.

For example:

```
Collection Room
→ Storage
→ Pharmacy
→ Lab Corridor
→ Pathology Lab
```

contains:

```
4 movements
```

4. Search Order

The sequence in which hospital locations are explored.

For example:

```
Collection Room
→ Reception
→ Emergency Desk
→ Storage
→ ...
```

This helps understand the behavior of each algorithm.

5. Success / Solution Status

The program reports:

```
Route Found
```

or:

```
Route Not Found
```

6. Number of Iterations

This is particularly useful for Hill Climbing.

The program allows up to:

```
20 iterations
```

7. Genetic Algorithm Fitness

The fitness is calculated as:

$$Fitness = \frac{1}{Cost + 1}$$

A lower-cost route has a higher fitness value.

8. Number of Generations

The Genetic Algorithm runs for:

```
30 generations
```

The progress of the best route is recorded during the generations.

---

17. Result Table

After running the program, the following table can be filled using actual output:

Algorithm	Category	Cost	Nodes/Steps	Status
BFS	Uninformed	Actual	Actual	Found/Not Found
DFS	Uninformed	Actual	Actual	Found/Not Found
DLS	Uninformed	Actual	Actual	Found/Not Found
IDS	Uninformed	Actual	Actual	Found/Not Found
Bidirectional	Uninformed	Actual	Actual	Found/Not Found
Greedy Best First	Informed	Actual	Actual	Found/Not Found
A*	Informed	Actual	Actual	Found/Not Found
Hill Climbing	Local	Actual	Actual	Found/Not Found
Genetic Algorithm	Local/Optimization	Actual	Actual	Found
Backtracking CSP	CSP	Actual	Actual	Found/Not Found

Note: Actual values should be copied from the output generated by the Python program.

---

18. Critical Analysis

The algorithms differ in the way they explore the hospital environment.

BFS provides systematic exploration and can produce a shortest route in terms of the number of edges when all movements have equal cost. However, it may explore many hospital locations before reaching the Pathology Lab and does not consider different edge costs.

DFS explores one branch deeply before backtracking. It may find a route quickly depending on the traversal order, but the route may contain more movements or higher transport cost than another available route.

DLS limits the search depth and therefore prevents exploration beyond the specified limit. However, if the depth limit is insufficient, the Pathology Lab may not be reached.

IDS combines the depth control of DLS with repeated searches at increasing depths. It is useful when the required route depth is unknown.

Bidirectional Search searches from both the Collection Room and the Pathology Lab. When the searches meet, the complete route can be constructed. This can reduce the effective search depth for suitable problems.

Greedy Best-First Search uses heuristic information to guide the search toward the Pathology Lab. It can reach the goal efficiently in some cases, but because it does not consider the transport cost already

incurred, the route found is not necessarily the lowest-cost route.

A* considers both actual transport cost and estimated remaining cost:

$$f(n) = g(n) + h(n)$$

Therefore, it provides a cost-aware approach to hospital sample route planning. With an appropriate admissible heuristic and cost model, A* can provide an optimal minimum-cost route.

Hill Climbing focuses on improving the current route instead of systematically exploring the entire search space. It requires relatively little memory but can become trapped in a local optimum.

The Genetic Algorithm maintains multiple candidate routes simultaneously. Selection, crossover, and mutation allow the algorithm to explore and improve candidate solutions. However, its results can vary because of its randomized operations.

Backtracking is useful when route selection must satisfy strict constraints. It rejects invalid decisions and returns to an earlier decision point when necessary.

Thus, different algorithms are suitable for different requirements. A hospital medical sample transportation system can use **path search, optimization, and constraint checking** together rather than relying on only one search strategy.

---

19. Advantages of the Proposed System

- Represents a practical hospital transportation application.
- Demonstrates multiple Artificial Intelligence search techniques.
- Represents hospital locations using a graph.
- Assigns transport costs to hospital connections.
- Can calculate transport route cost.
- Can identify feasible routes between hospital departments.
- Can incorporate heuristic information.
- Can optimize candidate routes.
- Can compare different search strategies.
- Provides an interactive Google Colab interface.
- Allows users to view the hospital map.
- Can be extended to larger hospital environments.
- Can be extended to real hospital navigation and transport systems in future implementations.

---

20. Limitations

The proposed implementation uses a simplified representation of a real hospital environment.

Real hospital sample transportation may involve:

- Moving patients and staff.
- Crowded corridors.
- Emergency situations.
- Elevators and stairs.
- Restricted medical areas.
- Infection-control zones.

- Time-sensitive samples.
- Multiple samples at the same time.
- Changing hospital conditions.
- Security restrictions.
- Real-time location information.

Therefore, the experimental results from the simulated hospital graph cannot directly represent the performance of a real hospital transportation robot or system.

The Genetic Algorithm and Hill Climbing results may also vary depending on their initial solutions and algorithm parameters.

The current implementation also uses a fixed hospital graph and fixed heuristic values rather than real-time hospital data.

---

21. Future Scope

The system can be improved by:

- Integrating real hospital floor maps.
- Using real-time location data.
- Incorporating GPS or indoor positioning systems.
- Adding dynamic obstacles such as patients, staff, and equipment.
- Including elevator and staircase information.
- Adding sample priority levels.
- Including sample delivery deadlines.
- Considering infection-control restrictions.
- Including transport personnel or robot battery consumption.
- Adding dynamic route re-planning.
- Using Random-Restart Hill Climbing.
- Improving Genetic Algorithm crossover and mutation techniques.
- Adding MRV and Forward Checking to the CSP.
- Comparing multiple heuristics for A*.
- Testing the system on larger hospital graphs.
- Integrating the system with an autonomous medical delivery robot.

A multi-objective cost function could also be introduced:

$$Cost = w_1(Transport\ Time) + w_2(Distance) + w_3(Battery) + w_4(Risk)$$

where the weights represent the relative importance of each factor.

For example, a medical sample with high priority may require a route that minimizes **transport time** rather than only physical distance.

---

22. Ethical and Societal Considerations

Automated medical sample transportation can improve efficiency and reduce repetitive transport work, but its deployment should consider **patient safety, sample integrity, privacy, and reliability**.

The system should ensure that medical samples are delivered to the correct laboratory destination.

The transport route should avoid unsafe areas and should not interfere with patients, healthcare workers, or emergency activities.

The system should also provide safe fallback behavior when the hospital environment changes, a route becomes unavailable, or no feasible route exists.

Medical samples may contain sensitive patient-related information. Therefore, any real implementation should follow appropriate privacy and data-security practices.

Automation can also change the responsibilities of hospital workers. Therefore, implementation should consider worker training, human oversight, system reliability, and responsible use of autonomous transportation technology.

---

23. Conclusion

This practical demonstrates how Artificial Intelligence search algorithms can be applied to a real-life **Hospital Medical Sample Transport** problem.

Uninformed algorithms such as **BFS, DFS, DLS, IDS, and Bidirectional Search** explore the hospital graph without heuristic information. Informed algorithms such as **Greedy Best-First Search and A*** use heuristic information to guide navigation toward the Pathology Lab.

Local search techniques such as **Hill Climbing and Genetic Algorithm** focus on improving candidate routes and are useful for optimization problems. **Backtracking Constraint Satisfaction** provides a systematic way to find routes while satisfying constraints such as valid hospital connections and avoiding repeated locations.

The practical demonstrates that the choice of search strategy depends on the characteristics of the problem. Systematic search is useful for route discovery, heuristic search can guide navigation toward the target, local search can optimize candidate routes, and constraint satisfaction can ensure that the selected route follows required rules.

```
# Hospital Medical Sample Transport Route Planner
# Problem Solving through Search - Practical Assignment 2

from collections import deque
import heapq
import random
import ipywidgets as widgets
from IPython.display import display, clear_output

START = "Collection Room"
GOAL = "Pathology Lab"

# Hospital map: edge value represents transport time/cost
graph = {
    "Collection Room": {"Reception": 3, "Emergency Desk": 5, "Storage": 4},
    "Reception": {"Collection Room": 3, "Waiting Area": 2, "Nurse Station": 4},
    "Emergency Desk": {"Collection Room": 5, "Nurse Station": 2, "ICU Corridor": 4},
    "Storage": {"Collection Room": 4, "Pharmacy": 3, "Lab Corridor": 5},
    "Waiting Area": {"Reception": 2, "Nurse Station": 3, "Lab Corridor": 6},
    "Nurse Station": {"Reception": 4, "Emergency Desk": 2, "Waiting Area": 3, "ICU Corridor": 3},
    "ICU Corridor": {"Emergency Desk": 4, "Nurse Station": 3, "Operating Wing": 4, "Lab Corridor":
    "Pharmacy": {"Storage": 3, "Lab Corridor": 4, "Radiology": 5},
    "Lab Corridor": {"Storage": 5, "Waiting Area": 6, "ICU Corridor": 5,
```

```
                      "Pharmacy": 4, "Radiology": 3, "Pathology Lab": 4},
    "Operating Wing": {"ICU Corridor": 4, "Radiology": 3},
    "Radiology": {"Pharmacy": 5, "Lab Corridor": 3, "Operating Wing": 3, "Pathology Lab": 5},
    "Pathology Lab": {"Lab Corridor": 4, "Radiology": 5}
}

heuristic = {
    "Collection Room": 18, "Reception": 16, "Emergency Desk": 15,
    "Storage": 14, "Waiting Area": 13, "Nurse Station": 12,
    "ICU Corridor": 9, "Pharmacy": 10, "Lab Corridor": 5,
    "Operating Wing": 7, "Radiology": 4, "Pathology Lab": 0
}

hospital_map = """
                HOSPITAL SAMPLE TRANSPORT MAP

 Collection Room
    |      |       |
 Reception Emergency Storage
    |       |       |
 Waiting  Nurse    Pharmacy
    |       |        |
    +-------+---- ICU Corridor
                   |       |
              Operating  Lab Corridor
                 |       |      |
              Radiology--+------+
                     |       |
                     +-- Pathology Lab

START : Collection Room
GOAL  : Pathology Lab
"""

def show_map():
    print(hospital_map)

def path_cost(path):
    if not path:
        return 0
    return sum(graph[path[i]][path[i + 1]] for i in range(len(path) - 1))

def valid_path(path):
    if not path or path[0] != START or path[-1] != GOAL:
        return False
    if len(path) != len(set(path)):
        return False
    return all(path[i + 1] in graph[path[i]] for i in range(len(path) - 1))

def bfs(start, goal):
    q = deque([(start, [start])])
    visited = set()
    order = []

    while q:
        node, path = q.popleft()
        if node in visited:
            continue
        visited.add(node)
        order.append(node)
```

```
        if node == goal:
            return path, order

        for nxt in graph[node]:
            if nxt not in visited:
                q.append((nxt, path + [nxt]))
    return None, order

def dfs(start, goal):
    stack = [(start, [start])]
    visited = set()
    order = []

    while stack:
        node, path = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        order.append(node)

        if node == goal:
            return path, order

        for nxt in reversed(list(graph[node])):
            if nxt not in visited:
                stack.append((nxt, path + [nxt]))
    return None, order

def dls(start, goal, limit):
    order = []

    def search(node, path, depth):
        order.append(node)
        if node == goal:
            return path
        if depth == limit:
            return None

        for nxt in graph[node]:
            if nxt not in path:
                result = search(nxt, path + [nxt], depth + 1)
                if result:
                    return result
        return None

    return search(start, [start], 0), order

def ids(start, goal):
    order = []
    for limit in range(1, len(graph) + 2):
        path, visited = dls(start, goal, limit)
        order.extend(visited)
        if path:
            return path, order
    return None, order

def bidirectional_search(start, goal):
    if start == goal:
        return [start], [start]

    front = deque([start])
```

```
    back = deque([goal])
    parent_f = {start: None}
    parent_b = {goal: None}
    order = []
    meet = None

    while front and back and meet is None:
        current = front.popleft()
        order.append(current)

        for nxt in graph[current]:
            if nxt not in parent_f:
                parent_f[nxt] = current
                front.append(nxt)
            if nxt in parent_b:
                meet = nxt
                break

        if meet:
            break

        current = back.popleft()
        order.append(current)

        for nxt in graph[current]:
            if nxt not in parent_b:
                parent_b[nxt] = current
                back.append(nxt)
            if nxt in parent_f:
                meet = nxt
                break

    if meet is None:
        return None, order

    left = []
    node = meet
    while node is not None:
        left.append(node)
        node = parent_f[node]
    left.reverse()

    right = []
    node = parent_b[meet]
    while node is not None:
        right.append(node)
        node = parent_b[node]

    return left + right, order

def greedy_search(start, goal):
    heap = [(heuristic[start], start, [start])]
    visited = set()
    order = []

    while heap:
        h, node, path = heapq.heappop(heap)
        if node in visited:
            continue
        visited.add(node)
        order.append(node)
```

```
        if node == goal:
            return path, order

        for nxt in graph[node]:
            if nxt not in visited:
                heapq.heappush(heap, (heuristic[nxt], nxt, path + [nxt]))

    return None, order

def a_star(start, goal):
    heap = [(heuristic[start], 0, start, [start])]
    best_g = {start: 0}
    visited = set()
    order = []

    while heap:
        f, g, node, path = heapq.heappop(heap)
        if node in visited:
            continue
        visited.add(node)
        order.append(node)

        if node == goal:
            return path, order

        for nxt, cost in graph[node].items():
            new_g = g + cost
            if new_g < best_g.get(nxt, float("inf")):
                best_g[nxt] = new_g
                heapq.heappush(
                    heap,
                    (new_g + heuristic[nxt], new_g, nxt, path + [nxt])
                )

    return None, order

def all_paths(node, goal, path=None):
    path = [node] if path is None else path
    if node == goal:
        return [path]

    routes = []
    for nxt in graph[node]:
        if nxt not in path:
            routes.extend(all_paths(nxt, goal, path + [nxt]))
    return routes

ALL_ROUTES = all_paths(START, GOAL)

def hill_climbing(start, goal):
    routes = [r for r in ALL_ROUTES if r[0] == start and r[-1] == goal]
    if not routes:
        return None, []

    current = max(routes, key=path_cost)
    history = [current.copy()]

    for _ in range(20):
        neighbors = []
        for route in routes:
```

```
            if route == current:
                continue
            common = 0
            for a, b in zip(current, route):
                if a != b:
                    break
                common += 1
            if common >= 2 and path_cost(route) < path_cost(current):
                neighbors.append(route)

        if not neighbors:
            break

        current = min(neighbors, key=path_cost).copy()
        history.append(current.copy())

    return current, history

def fitness(route):
    return 1 / (path_cost(route) + 1) if valid_path(route) else 0

def genetic_algorithm(pop_size=20, generations=30, mutation_rate=0.20, seed=7):
    rng = random.Random(seed)
    population = [rng.choice(ALL_ROUTES).copy() for _ in range(pop_size)]
    progress = []
    best = min(population, key=path_cost).copy()

    for generation in range(generations):
        current = min(population, key=path_cost)
        if path_cost(current) < path_cost(best):
            best = current.copy()

        progress.append((generation + 1, best.copy(), path_cost(best)))

        ranked = sorted(population, key=fitness, reverse=True)
        parents = ranked[:max(2, pop_size // 2)]
        new_population = [best.copy()]

        while len(new_population) < pop_size:
            p1, p2 = rng.choice(parents), rng.choice(parents)
            common = [n for n in p1[1:-1] if n in p2[1:-1]]

            if common:
                meet = rng.choice(common)
                child = p1[:p1.index(meet)] + p2[p2.index(meet):]
                if not valid_path(child):
                    child = rng.choice([p1, p2]).copy()
            else:
                child = rng.choice([p1, p2]).copy()

            if rng.random() < mutation_rate:
                child = rng.choice(ALL_ROUTES).copy()

            new_population.append(child)

        population = new_population

    return best, path_cost(best), progress

def csp_route(start, goal):
    path = []
```

```
    path = []
    visited = set()
    order = []

    def backtrack(node):
        path.append(node)
        visited.add(node)
        order.append(node)

        if node == goal:
            return True

        # Constraints: valid hospital path, no repeated location,
        # and avoid returning to already occupied nodes.
        for nxt in graph[node]:
            if nxt not in visited and nxt in graph:
                if backtrack(nxt):
                    return True

        path.pop()
        visited.remove(node)
        return False

    return (path if backtrack(start) else None), order

def display_result(name, path, order):
    print("\n" + "=" * 72)
    print(name)
    print("=" * 72)

    if path:
        print("Status        : Route Found")
        print("Route         :", " -> ".join(path))
        print("Transport Cost:", path_cost(path), "units")
    else:
        print("Status        : Route Not Found")

    print("Nodes/Steps   :", len(order))
    print("Search Order  :", " -> ".join(order))

def run_algorithm(choice):
    if choice == "BFS":
        path, order = bfs(START, GOAL)
    elif choice == "DFS":
        path, order = dfs(START, GOAL)
    elif choice == "DLS":
        path, order = dls(START, GOAL, 8)
    elif choice == "IDS":
        path, order = ids(START, GOAL)
    elif choice == "Bidirectional":
        path, order = bidirectional_search(START, GOAL)
    elif choice == "Greedy Best First":
        path, order = greedy_search(START, GOAL)
    elif choice == "A*":
        path, order = a_star(START, GOAL)
    elif choice == "Hill Climbing":
        path, history = hill_climbing(START, GOAL)
        display_result(choice, path, history)
        if path:
            print("\nImprovement History:")
            for i, route in enumerate(history, 1):
```

```
                print(f"Step {i}: {' -> '.join(route)} | Cost: {path_cost(route)}")
        return
    elif choice == "Genetic Algorithm":
        path, cost, progress = genetic_algorithm()
        print("\n" + "=" * 72)
        print("GENETIC ALGORITHM")
        print("=" * 72)
        print("Status        : Route Found")
        print("Best Route    :", " -> ".join(path))
        print("Best Cost     :", cost, "units")
        print("Generations   :", len(progress))
        for gen, route, route_cost in progress:
            if gen == 1 or gen % 5 == 0 or gen == len(progress):
                print(f"Generation {gen}: {route_cost} units")
        return
    elif choice == "Backtracking CSP":
        path, order = csp_route(START, GOAL)

    display_result(choice, path, order)

def compare_all():
    results = []

    algorithms = [
        ("BFS", lambda: bfs(START, GOAL)),
        ("DFS", lambda: dfs(START, GOAL)),
        ("DLS", lambda: dls(START, GOAL, 8)),
        ("IDS", lambda: ids(START, GOAL)),
        ("Bidirectional", lambda: bidirectional_search(START, GOAL)),
        ("Greedy Best First", lambda: greedy_search(START, GOAL)),
        ("A*", lambda: a_star(START, GOAL)),
        ("Hill Climbing", lambda: hill_climbing(START, GOAL)),
        ("Genetic Algorithm", lambda: genetic_algorithm()),
        ("Backtracking CSP", lambda: csp_route(START, GOAL))
    ]

    for name, function in algorithms:
        result = function()

        if name == "Genetic Algorithm":
            path, cost, progress = result
            steps = len(progress)
        elif name == "Hill Climbing":
            path, history = result
            cost = path_cost(path)
            steps = len(history)
        else:
            path, order = result
            cost = path_cost(path)
            steps = len(order)

        results.append((name, cost, steps, "Found" if path else "Not Found"))

    print("\n" + "=" * 78)
    print("HOSPITAL MEDICAL SAMPLE TRANSPORT - PERFORMANCE COMPARISON")
    print("=" * 78)
    print(f"{'Algorithm':<24}{'Cost':<12}{'Nodes/Steps':<15}{'Status'}")
    print("-" * 78)

    for name, cost, steps, status in results:
        print(f"{name:<24}{cost:<12}{steps:<15}{status}")
```

```
    print("-" * 78)

def run_clicked(_):
    with output:
        clear_output()
        run_algorithm(algorithm_dropdown.value)

def compare_clicked(_):
    with output:
        clear_output()
        compare_all()

def map_clicked(_):
    with output:
        clear_output()
        show_map()

def exit_clicked(_):
    with output:
        clear_output()
        print("Hospital Medical Sample Transport Planner closed.")

title = widgets.HTML(
    value="""
    <div style="
        background:#EAF3F8;
        border-left:8px solid #1565C0;
        padding:18px;
        border-radius:10px;
        margin-bottom:12px;">
        <h1 style="margin:0;color:#154360;">Hospital Sample Route Planner</h1>
        <p style="margin:6px 0 0;color:#566573;">
            Search Algorithm Performance Evaluation
        </p>
    </div>
    """
)

section = widgets.HTML(
    value="""
    <div style="
        background:#F8F9F9;
        padding:10px 14px;
        border-radius:7px;
        border:1px solid #D5DBDB;">
        <b>📍 Route:</b> Collection Room   →   Pathology Lab
    </div>
    """
)

algorithm_dropdown = widgets.Dropdown(
    options=[
        "BFS", "DFS", "DLS", "IDS", "Bidirectional",
        "Greedy Best First", "A*", "Hill Climbing",
        "Genetic Algorithm", "Backtracking CSP"
    ],
    description="Select Search:",
    layout=widgets.Layout(width="430px"),
    style={"description_width": "initial"}
)
```

```
run_button = widgets.Button(
    description="RUN SEARCH",
    button_style="success",
    icon="play",
    layout=widgets.Layout(width="150px", height="38px")
)

compare_button = widgets.Button(
    description="Compare Results",
    button_style="info",
    icon="bar-chart",
    layout=widgets.Layout(width="170px", height="35px")
)

map_button = widgets.Button(
    description="View Hospital Map",
    button_style="warning",
    icon="map",
    layout=widgets.Layout(width="170px", height="35px")
)

exit_button = widgets.Button(
    description="Clear",
    button_style="danger",
    icon="refresh",
    layout=widgets.Layout(width="120px", height="35px")
)

output = widgets.Output()

def run_clicked(_):
    with output:
        clear_output()
        run_algorithm(algorithm_dropdown.value)

def compare_clicked(_):
    with output:
        clear_output()
        compare_all()

def map_clicked(_):
    with output:
        clear_output()
        show_map()

def exit_clicked(_):
    with output:
        clear_output()
        print("Output cleared. Select an option to continue.")

run_button.on_click(run_clicked)
compare_button.on_click(compare_clicked)
map_button.on_click(map_clicked)
exit_button.on_click(exit_clicked)

display(title)
display(section)
display(widgets.HBox(
    [algorithm_dropdown, run_button],
    layout=widgets.Layout(
```

```
        padding="12px 0",
        align_items="center"
    )
))

display(widgets.HTML(
    "<b>Analysis Tools</b>"
))

display(widgets.HBox(
    [compare_button, map_button, exit_button],
    layout=widgets.Layout(
        gap="10px",
        padding="8px 0 14px 0"
    )
))

display(output)
```

Hospital Sample Route Planner

Search Algorithm Performance Evaluation

📍 **Route:** Collection Room → Pathology Lab

Select Search: BFS RUN SEARCH

Analysis Tools

Compare Results View Hospital Map Clear

```
======================================================================
BFS
======================================================================
Status        : Route Found
Route         : Collection Room -> Storage -> Lab Corridor -> Pathology Lab
Transport Cost: 13 units
Nodes/Steps   : 12
Search Order  : Collection Room -> Reception -> Emergency Desk -> Storage -> Waiting Area ->
Nurse Station -> ICU Corridor -> Pharmacy -> Lab Corridor -> Operating Wing -> Radiology ->
Pathology Lab
```